REPRESENTACIÓN EFICIENTE DE DATOS Y ESTRUCTURAS DISPERSAS

¿Qué es representación eficiente de datos?

Consiste en organizar y almacenar información aprovechando bien los recursos del computador. No basta con guardar bien, tambien importa el costo de usar los datos.

Memoria: Cuanto espacio ocupa los datos.

Procesamiento: Que tan rapido se consultan, recorren o modifican.

OBJETIVO: Evitar desperdiciar recursos cuando gran parte de la información está vacía.



MATRICES

¿QUÉ ES UNA MATRIZ?

Una matriz organiza datos en fila y columnas. una matriz de m filas y n columnas tiene m *n posiciones. 3 filas * 4 columnas = 12 posiciones.

Imaginemos una plataforma de películas donde los usuarios califican las películas que han visto. Si tenemos 1.000 usuarios y 5.000 películas, la matriz tendría 5.000.000 de posiciones, pero cada usuario solo habrá calificado una pequeña parte de las películas. Muchas posiciones quedan vacías o en ceros

```
Unity Message | 0 references
void Start()
{
    rows = 3;
    columns = 2;

    float[,] matA = new float[rows, colu
    matA[0, 0] = 3f;
    matA[0, 1] = -1f;
    matA[1, 0] = -2.5f;
    matA[1, 1] = 4f;
    matA[2, 0] = -5f;
```



Matrices densas y dispersas

MATRIZ DENSA

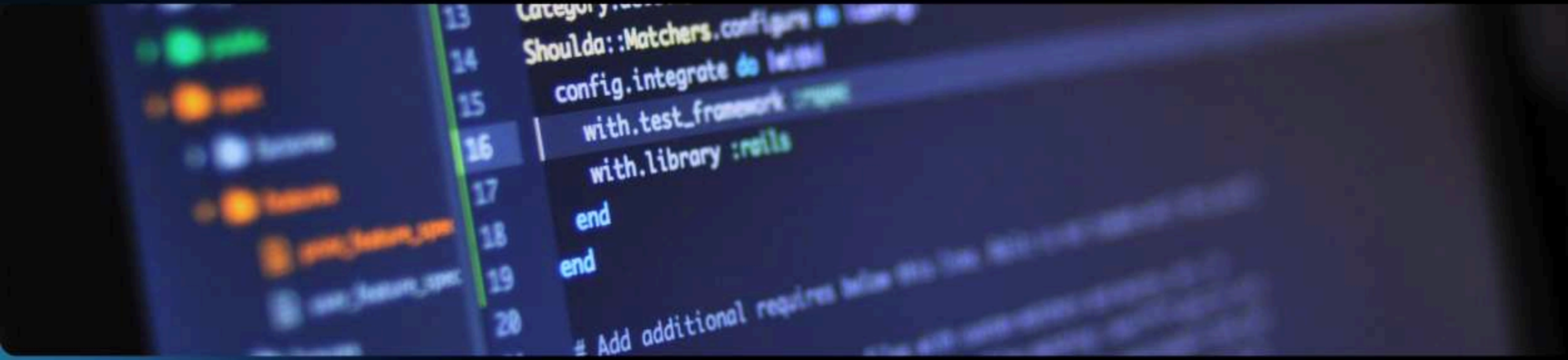
- Almacena todos los elementos, imcluidos el 0.
- Es sencilla de entender.
- Puede convenir si en gran parte de las posiciones tiene valores útiles.

MATRIZ DISPERSA

- La mayoría de las posiciones contienen cero o no tienen un dato registrado.
- Guarda los valores no nulos y la información necesaria para ubicarlos.
- Busca evitar desperdicio de memoria.

Densa = acceso directo sencillo

Dispersa = prioriza ahorro de memoria.



Representación mediante tripletas (COO)

COO SIGNIFICA COORDINATE FORMAT

Una forma sencilla de representar una matriz dispersa es guardar cada valor no nulo como una tripleta: (fila, columna, valor).

Ventajas: es intuitivo, fácil de construir y útil para reunir entradas dispersas.
Limitaciones: para algunas operaciones repetidas o acceso por filas puede resultar menos conveniente que otros formatos.

CRS: almacenamiento comprimido por filas

CRS SIGNIFICA ALMACENAMIENTO COMPRIMIDO POR FILAS.

Organiza los valores no nulos por filas y usa tres arreglos.

- Data: contiene los valores no nulos, ordenados por filas.
- índices: contiene el índice de columna correspondiente a cada valor de data.
- indptr: indica dónde comienza cada fila y dónde termina dentro de data. Tiene una posición más que la cantidad de filas.

CSR es útil para recorrer filas y realizar operaciones matemáticas, insertar elementos uno por uno.



Comparación entre densa, COO y CRS

Representación	Cómo guarda datos	Ventaja	Limitación
Densa	Todas las posiciones, incluidos ceros	Acceso directo sencillo	Puede gastar mucha memoria si hay muchos ceros
COO / tripletas	Fila, columna y valor por cada entrada no nula	Fácil de construir y entender	No es ideal para todas las operaciones
CSR	Valores, columnas y punteros de filas	Compacta y eficiente para recorrer filas	Inserciones individuales pueden ser costosas

La elección depende de los datos y de las operaciones que se necesiten.

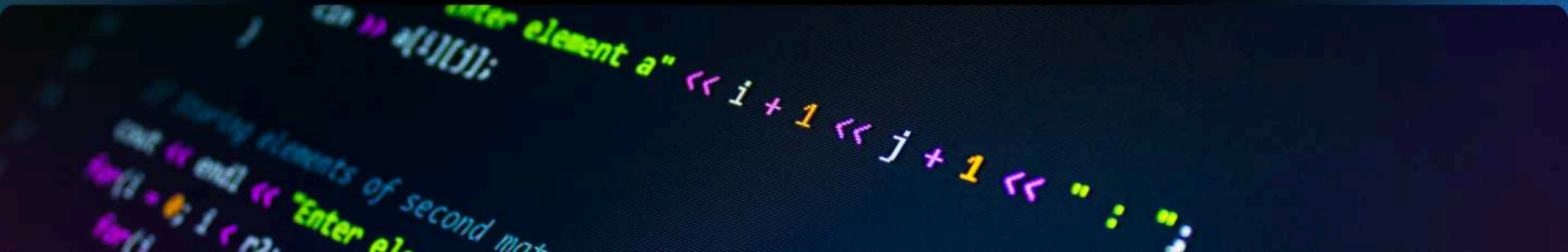
Fundamento matemático

MEMORIA

Si una matriz densa tiene m filas y n columnas, almacena m × n elementos. Si cada elemento ocupa b bytes, la memoria aproximada es:
Memoria densa ≈ m × n × b bytes. Ejemplo tenemos 1000 × 1000 × 8 = 8.000.000 bytes

Ahorro porcentual estimado = (1 – memoria CSR / memoria densa) × 100.

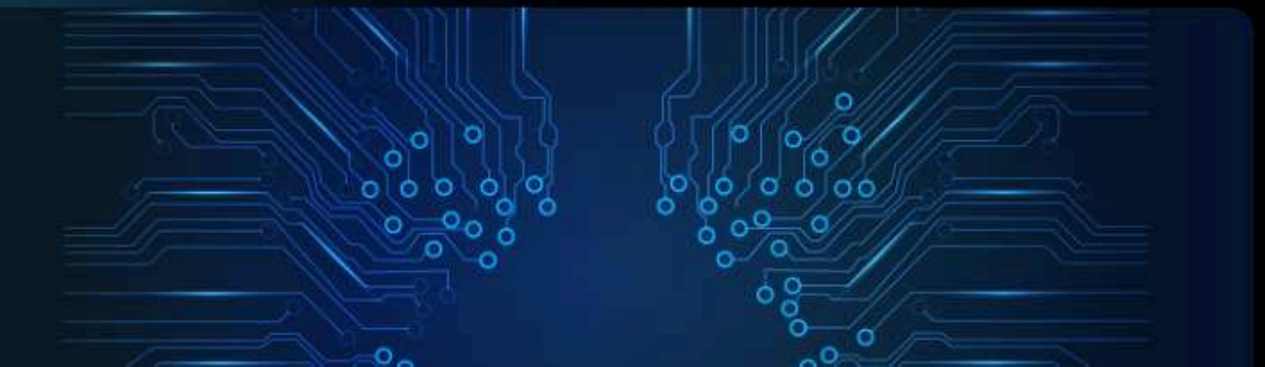
Los valores exactos debe medirse dependiendo de los valores e índices almacenados.

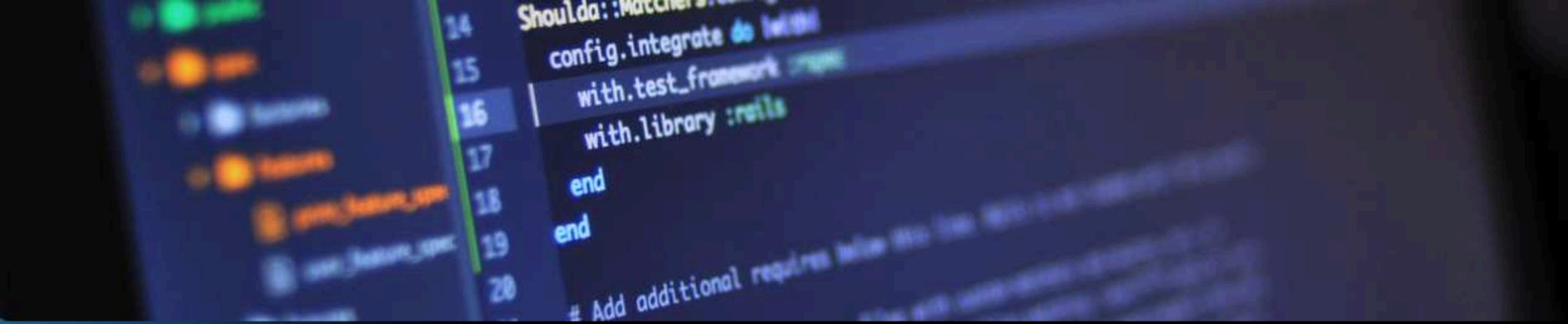


COMPLEJIDAD TEMPORAL Y OPERACIONES

La complejidad describe cómo crece el trabajo del algoritmo al aumentar el tamaño de los datos.

- ·Recorrer una matriz densa: O(m × n), se visitan todas las posiciones.
- ·Recorrer los valores almacenados en CSR: O(nnz), se recorren los valores no nulos.
- Consultar un elemento: En una matriz densa, el acceso por índices suele ser O(1). En CSR, depende de la búsqueda dentro de la fila.
- Construcción COO o LIL pueden ser más cómodos. Despues de construir, CRS suele convenir para operaciones





Conclusiones

La representación eficiente considera tanto memoria como tiempo de procesamiento.

COO guarda cada valor no nulo como (fila, columna,valor).

CSR almacena valores, índices de columna y punteros de filas.

La mejor representación depende de la cantidad de ceros y de las operaciones requeridas.

GRACIAS